

PRACTICE EXERCISES OF THE MICROPROCESSORS & MICROCONTROLLERS

Instructor: The Tung Than

Student's name: Nguyễn Đông Quân

Student code: 24521438

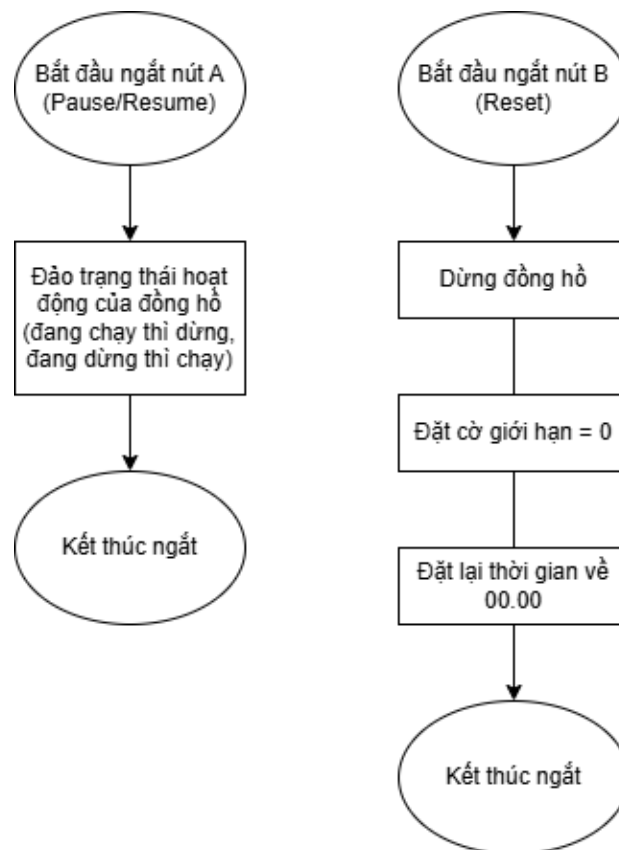
PRACTICE REPORT NO #3:

USING INTERRUPT

I. Content 1

Present and draw a flowchart to handle 2 buttons with the following functions:

- Button A: Pause/Resume stopwatch
- Button B: Reset the stopwatch.

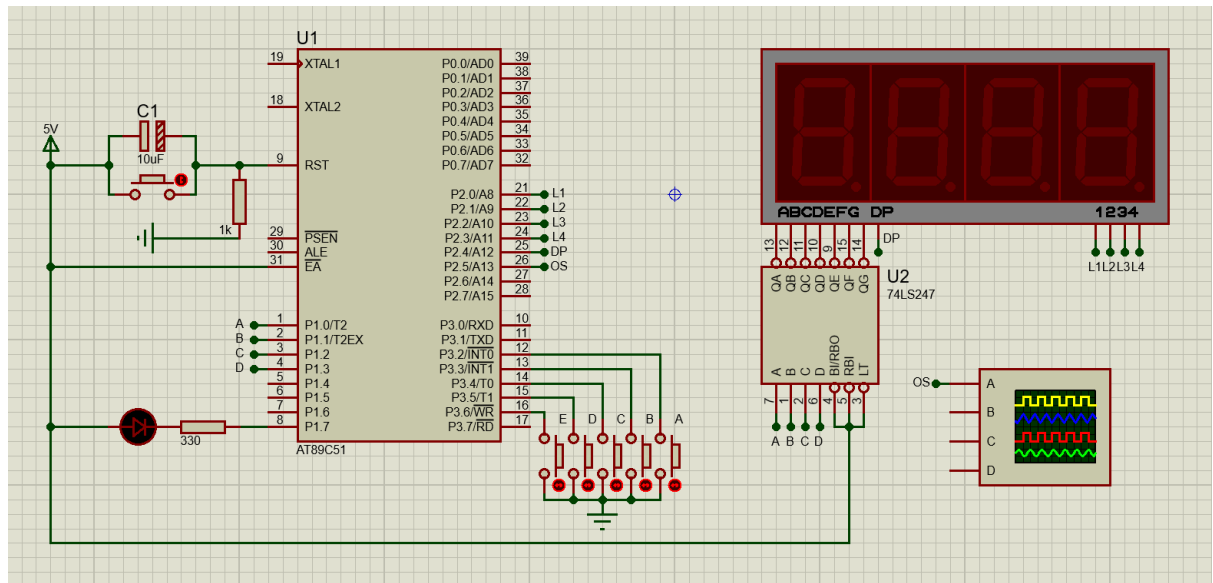


II. Content 2

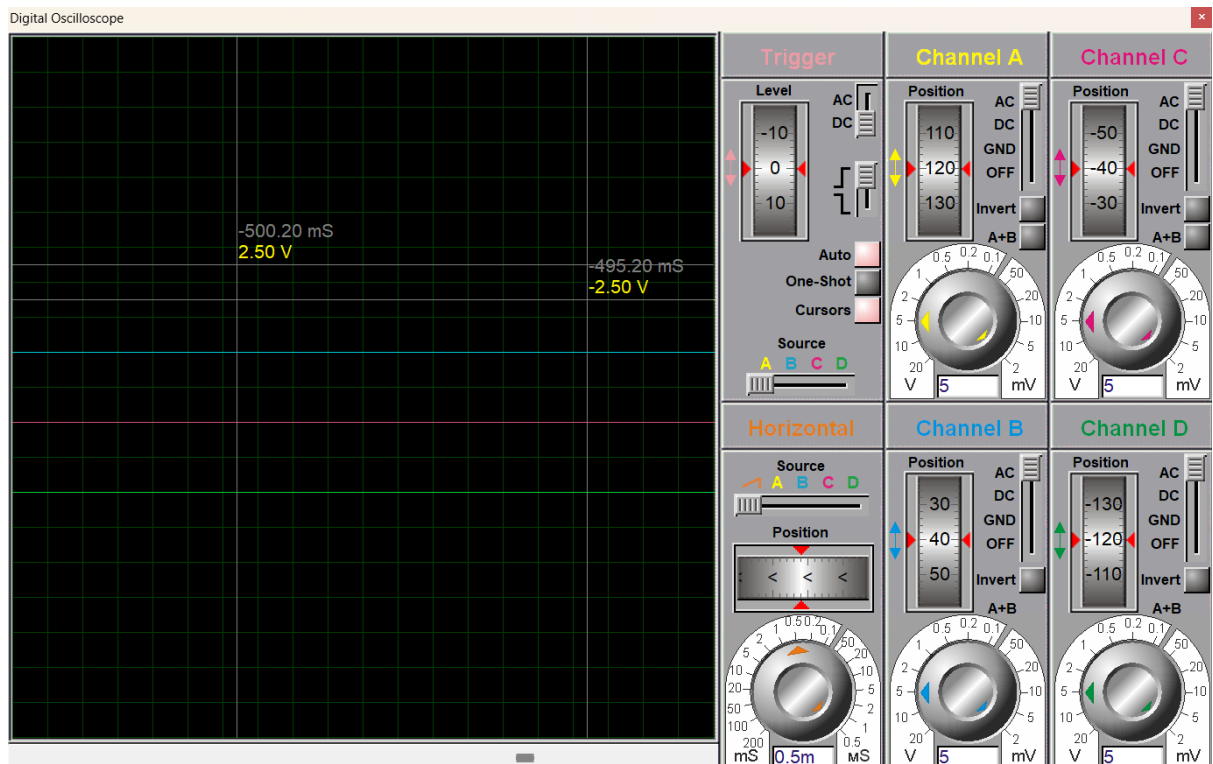
Using AT89C51/AT89C52 in combination with 4 7-Segment LED modules and 2 buttons above, design a Sport clock circuit with the ability to count accurately to 1% of seconds, counting range from 00.00 seconds to 99.99 seconds and has 2 buttons to control Pause/Resume and Reset. Add 2 buttons to the Sport watch with the following functions:

- Button C: Increase the number of seconds counting to 1 second (1 point)
- Button D: Decrease the number of seconds to 1 second (1 point)
- Button E: Switch the stopwatch speed between normal speed (1×) and double speed (2×).

When the stopwatch reaches 99.99 seconds, it automatically stops and a LED connected to P1.7 turns on. Pressing the Reset button clears the counter and returns the display to 00.00 seconds.



Mạch stopwatch sử dụng thêm Interrupt



Kiểm chứng chu kỳ quét LED (5ms) trên Oscilloscope

Source code:

```
;=====
; Main.asm file generated by New Project wizard
; Processor: AT89C51
; Compiler: ASEM-51 (Proteus)
;=====

$NOMOD51
$INCLUDE (8051.MCU)

;=====
; DEFINITIONS
;=====

    flag_RUN BIT 20H.0; Run flag
    flag_TUP BIT 20H.1; Time update flag
    flag_SPD BIT 20H.2; Speed flag
    flag_STP BIT 20H.4; Stop flag

;=====
; VARIABLES
;=====

    MSEC EQU 30H; Milliseconds
    SEC EQU 31H; Seconds
    TCNT EQU 32H; Timer counter
    TREL EQU 33H; Timer reload
    SCAN_PTR EQU 34H; LED buffer pointer

;=====
; RESET AND INTERRUPT VECTORS
;=====

    ORG 0000H
    SJMP Start

    ORG 0003H; INT0 pause/resume
    CPL flag_RUN
```

RETI

ORG 000BH; Timer0 5ms

LJMP T0_ISR

ORG 0013H; INT1 reset

CLR flag_RUN

CLR flag_STP

MOV SEC, #0

MOV MSEC, #0

LCALL Update_Buffer; Update immediately

RETI

;
=====

; CODE SEGMENT

;
=====

ORG 0030H

Start:

MOV SP, #50H

MOV TMOD, #01H

MOV TH0, #0ECH; Reload MSB for 12MHz (5ms)

MOV TL0, #078H; Reload LSB

SETB TR0

MOV IE, #87H; EA, EX0, ET0, EX1

SETB IT0; Edge trigger INT0

SETB IT1; Edge trigger INT1

CLR A

MOV 20H, A; Clear all flags

MOV MSEC, A

MOV SEC, A

MOV SCAN_PTR, #40H; Point to first LED data

MOV TREL, #2

MOV TCNT, #2

CLR P2.5; Init oscilloscope pin

; Init Port 2 LED selection masks in RAM

```
MOV 41H, #11H; Mask for LED 1
MOV 43H, #02H; Mask for LED 2 (with DP)
MOV 45H, #14H; Mask for LED 3
MOV 47H, #18H; Mask for LED 4
```

```
LCALL Update_Buffer; First display render
```

Loop:

```
JB flag_TUP, Do_Time
JNB P3.4, BtnC
JNB P3.5, BtnD
JNB P3.6, BtnE
SJMP Loop
```

Do_Time:

```
CLR flag_TUP
MOV A, MSEC
ADD A, #1
DA A
MOV MSEC, A
CJNE A, #00H, Time_Update_Dis
```

```
MOV A, SEC
ADD A, #1
DA A
MOV SEC, A
CJNE A, #00H, Time_Update_Dis
```

```
MOV SEC, #99H; Max hit
MOV MSEC, #99H
CLR flag_RUN
SETB flag_STP
```

Time_Update_Dis:

```
LCALL Update_Buffer
SJMP Loop
```

BtnC;; +1s

```
JNB P3.4, $; Wait for release
MOV A, SEC
CJNE A, #99H, Add_1s
SJMP Loop
```

Add_1s:

```
ADD A, #1
DA A
MOV SEC, A
LCALL Update_Buffer
SJMP Loop
```

BtnD:: -1s

```
JNB P3.5, $; Wait for release
MOV A, SEC
JZ Loop; Skip if 0
ADD A, #99H; Add 99 is sub 1 in BCD
DA A
MOV SEC, A
LCALL Update_Buffer
SJMP Loop
```

BtnE:: Toggle speed

```
JNB P3.6, $; Wait for release
CPL flag_SPD
MOV TREL, #2
JNB flag_SPD, Loop
MOV TREL, #1
SJMP Loop
```

; Display buffer (main loop)

Update_Buffer:

```
; SEC tens
MOV A, SEC
SWAP A
ANL A, #0FH
MOV 40H, A
```

```
; SEC units
MOV A, SEC
ANL A, #0FH
MOV 42H, A
```

```
; MSEC tens
MOV A, MSEC
SWAP A
ANL A, #0FH
MOV 44H, A
```

```
; MSEC units
MOV A, MSEC
ANL A, #0FH
MOV 46H, A
```

```
; Handle STOP LED on P1.7
JNB flag_STP, Clr_Stop_LED
RET; flag_STP=1: bit7=0 -> LED ON
```

Clr_Stop_LED:

```
ORL 40H, #80H; bit7=1 -> LED OFF
ORL 42H, #80H
ORL 44H, #80H
ORL 46H, #80H
RET
```

; TIMER 0 ISR

T0_ISR:

```
PUSH ACC
PUSH PSW
CPL P2.5;
```

```
; Reload timer while running
MOV A, TL0
ADD A, #078H; Period LSB for 12MHz 5ms
```

```

MOV TL0, A
MOV A, TH0
ADDC A, #0ECH; Period MSB
MOV TH0, A

; Ghosting fix
ANL P2, #0E0H
MOV R0, SCAN_PTR
MOV P1, @R0; Output digit data
INC R0
MOV A, @R0; Load LED select mask
ORL P2, A; Enable current LED
INC R0
CJNE R0, #48H, T0_Ptr
MOV R0, #40H

T0_Ptr:
MOV SCAN_PTR, R0
JNB flag_RUN, T0_End
DJNZ TCNT, T0_End
MOV TCNT, TREL
SETB flag_TUP

T0_End:
POP PSW
POP ACC
RETI

END

```

Giải thích thiết kế:

- **Vi điều khiển:** AT89C51 sử dụng thạch anh 12MHz.
- **Cấu trúc Port:**
 - **Port 1 (P1):** Đóng vai trò là bus dữ liệu. P1.0 đến P1.3 xuất trực tiếp mã BCD tới các chân ngõ vào của IC giải mã 74LS247. P1.7: Điều khiển LED đỏ đóng vai trò báo hiệu chạm mốc giới hạn thời gian theo yêu cầu đề bài.
 - **Port 2 (P2):** Đóng vai trò là bus điều khiển vị trí. Các chân P2.0 đến P2.3 xuất tín hiệu chọn LED (L1, L2, L3, L4) để cấp nguồn luân phiên cho 4 LED 7 đoạn.

Chân P2.4 điều khiển riêng biệt dấu chấm thập phân (DP). Chân P2.5 được thiết lập làm ngõ ra tín hiệu test, dùng để cắm vào Oscilloscope.

- **Port 3 (P3):** P3.2 đến P3.6 được cấu hình làm ngõ vào, đọc tín hiệu từ 5 nút điều khiển (A, B, C, D, E).

➤ **Logic điều khiển:**

- **Giải mã phần cứng:** Vì điều khiển chỉ đảm nhận việc xử lý logic thời gian và xuất mã BCD 4-bit. Việc dịch sang mã hiển thị 7 đoạn được giao cho IC giải mã 74LS247 để giảm tải xử lý cho CPU.
- **Quét LED động:** Tại một thời điểm, phần mềm chỉ xuất mã BCD của một con số ra Port 1, đồng thời bật đúng 1 chân tương ứng ở Port 2 (L1-L4) để cấp nguồn cho con số đó sáng. Quá trình này được đặt trong Ngắt Timer 0 lặp lại với chu kỳ cố định 5ms/LED và đạt tần số làm mới 50Hz cho toàn bộ 4 LED, tạo hiện tượng lưu ảnh giúp mắt người thấy cả 4 số đang sáng đồng thời một cách mượt mà, không bị nhấp nháy.

➤ **Tính toán điện trở sử dụng:** 330 Ohm cho LED đỏ và 999 Ohm cho mạch Reset.

- **Trở 330 Ohm:** Vì $I = \frac{V_{cc} - V_f}{R} = \frac{5 - 2.2}{330} \approx 8.5 \text{ mA} \rightarrow \text{LED sáng an toàn.}$
- **Trở 1k Ohm:** Trở kéo xuống đủ mạnh để thực hiện reset cứng. $I = \frac{V_{cc}}{R} = \frac{5}{1000} = 5 \text{ mA} \rightarrow \text{dòng trong ngưỡng an toàn.}$

Phần 1: Khai báo các cờ và biến

```
;=====
; DEFINITIONS
;=====

flag_RUN BIT 20H.0; Run flag
flag_TUP BIT 20H.1; Time update flag
flag_SPD BIT 20H.2; Speed flag
flag_STP BIT 20H.4; Stop flag

;=====
; VARIABLES
;=====

MSEC EQU 30H; Milliseconds
SEC EQU 31H; Seconds
TCNT EQU 32H; Timer counter
TREL EQU 33H; Timer reload
```

- **Vùng nhớ BIT (từ 20H trở đi):** Khởi tạo các cờ báo hiệu trạng thái:
 - flag_RUN: Cờ báo hiệu đồng hồ đang chạy (1) hay đang tạm dừng (0).
 - flag_TUP: Cờ báo hiệu đã đủ thời gian cập nhật chưa. ISR set flag_TUP lên 1 để báo cho chương trình chính tăng MSEC.
 - flag_SPD: Cờ tốc độ. 0 là chạy bình thường (1x), 1 là chạy nhanh gấp đôi (2x).
 - flag_STP: Cờ báo hiệu đã đếm đến giới hạn 99.99s.
- **Vùng nhớ BYTE (từ 30H trở đi):** Các biến lưu trữ giá trị 8-bit.
 - MSEC và SEC: Lưu giá trị phần trăm giây và giây (00-99) dưới dạng mã BCD.
 - TCNT và TREL: TCNT là biến đếm lùi số lần ngắt, TREL là giá trị nạp lại. VD: ngắt 5ms, nạp TREL=2 thì TCNT giảm 2 lần sẽ được 10ms).
 - SCAN_PTR: là con trỏ trỏ vào buffer LED trong RAM.

Phần 2: Khai báo vector ngắt

```

=====
;
; RESET AND INTERRUPT VECTORS
;
=====

    ORG 0000H
    SJMP Start

    ORG 0003H; INT0 pause/resume
    CPL flag_RUN
    RETI

    ORG 000BH; Timer0 5ms
    LJMP T0_ISR

    ORG 0013H; INT1 reset
    CLR flag_RUN
    CLR flag_STP
    MOV SEC, #0
    MOV MSEC, #0
    LCALL Update_Buffer; Update immediately
    RETI

```

Khi có một sự kiện xảy ra, CPU sẽ tự động ngắt công việc hiện tại và nhảy đến đúng địa chỉ tương ứng để xử lý.

- **ORG 0000H (Reset):** Nơi CPU bắt đầu chạy khi vừa cấp điện. Lệnh SJMP Start đóng vai trò điều hướng, đưa CPU nhảy qua vùng chứa các vector ngắt để đi vào chương trình chính.
- **ORG 0003H (INT0 – P3.2):** Xử lý nút nhấn Pause/Resume. Lệnh CPL flag_RUN đảo trạng thái cờ hoạt động một cách nhanh chóng (đang chạy thì dừng, đang dừng thì chạy), sau đó thoát ngắt ngay bằng lệnh RETI.
- **ORG 000BH (Timer0):** Xử lý tác vụ quét LED mỗi 5ms, lệnh LJMP T0_ISR được sử dụng để điều hướng CPU đến chương trình quét LED ở vùng nhớ tự do (0030H trở đi).
- **ORG 0013H (INT1 – P3.3):** Xử lý nút nhấn Reset. Thực hiện xóa cờ chạy (flag_RUN) và cờ báo động (flag_STP), xóa các biến đếm thời gian (SEC, MSEC) về 0. Đồng thời, tích hợp lệnh LCALL Update_Buffer để ép màn hình LED cập nhật con số 00.00 ngay lập tức, triệt tiêu độ trễ hiển thị trước khi thoát ngắt.

Phần 3: Khởi tạo hệ thống

```
;=====
; CODE SEGMENT
;=====

        ORG 0030H
Start:
        MOV SP, #50H
        MOV TMOD, #01H
        MOV TH0, #0ECH; Reload MSB for 12MHz (5ms)
        MOV TL0, #078H; Reload LSB
        SETB TR0
        MOV IE, #87H; EA, EX0, ET0, EX1
        SETB IT0; Edge trigger INT0
        SETB IT1; Edge trigger INT1

        CLR A
        MOV 20H, A; Clear all flags
        MOV MSEC, A
        MOV SEC, A
        MOV SCAN_PTR, #40H; Pointer to first LED data
        MOV TREL, #2
```

```

MOV TCNT, #2
CLR P2.5; Init oscilloscope pin

; Init Port 2 LED selection masks in RAM
MOV 41H, #11H; Mask for LED 1
MOV 43H, #02H; Mask for LED 2 (with DP)
MOV 45H, #14H; Mask for LED 3
MOV 47H, #18H; Mask for LED 4

LCALL Update_Buffer; First display render

```

- **ORG 0030H và MOV SP, #50H:** Chương trình chính đặt tại 0030H vì các địa chỉ trước đó (0000H–002FH) đã bị chiếm bởi bảng vector ngắt. Stack đặt tại 50H để tránh đè lên buffer LED ở 40H–47H, nếu để thấp hơn mỗi lần PUSH sẽ ghi đè lên dữ liệu hiển thị.
- **Thiết lập timer và ngắt:**
 - Timer 0 mode 1 là bộ đếm 16-bit, đếm từ giá trị nạp vào đến 0xFFFF rồi tràn và sinh ngắt. Với thạch anh 12MHz, mỗi chu kỳ máy = 1μs, nên nạp 65536 – 5000 = 60536 = 0xEC78 để tràn đúng sau 5000μs = 5ms.
 - IE = 87H = 1000 0111b bật đúng 3 ngắt cần dùng: INT0 (pause/resume), Timer 0 (scan LED + đếm giờ), INT1 (reset). Cấu hình SETB IT0 và SETB IT1 nghĩa là ngắt chỉ kích khi nút vừa được nhấn xuống (sườn âm).
- **Dọn dẹp hệ thống:** Đảm bảo đồng hồ luôn bắt đầu từ 00.00 và không có cờ rác nào đang bật khi khởi động.
- **Khởi tạo cấu trúc RAM xen kẽ:** Mỗi LED chiếm 2 ô liên tiếp: ô chẵn chứa dữ liệu BCD gửi ra P1, ô lẻ chứa mask P2 để chọn LED đó sáng. ISR chỉ cần tăng con trỏ R0 liên tục mà không cần logic rẽ nhánh.

RAM: 40H	41H	42H	43H	44H	45H	46H	47H
[digit]	[mask]	[digit]	[mask]	[digit]	[mask]	[digit]	[mask]
LED1	LED1	LED2	LED2	LED3	LED3	LED4	LED4

Các mask này được ghi một lần duy nhất khi khởi động và không bao giờ thay đổi nên chỉ có các ô chẵn (digit data) mới được cập nhật liên tục bởi Update_Buffer.

- **MOV TREL/TCNT, #2:** Timer ngắt mỗi 5ms. Mỗi lần ngắt, ISR chạy DJNZ TCNT, chỉ khi TCNT đếm ngược về 0 mới set flag_TUP và nạp lại TCNT = TREL. Với TREL = 2, phải qua 2 lần ngắt ($2 \times 5\text{ms} = 10\text{ms}$) thì main loop mới tăng MSEC một lần. 100 lần tăng $\times 10\text{ms} = 1000\text{ms} = 1\text{ giây}$, khớp đúng với dải BCD 00→99. Khi bấm BtnE, TREL đổi thành 1 → mỗi 5ms tăng MSEC một lần → đồng hồ chạy nhanh gấp đôi.

- **LCALL Update_Buffer:** Gọi ngay trước khi vào vòng lặp chính để các ô 40H, 42H, 44H, 46H có dữ liệu hợp lệ (0) ngay từ đầu. Nếu bỏ dòng này, ISR sẽ đọc rác từ RAM chưa khởi tạo và hiển thị số ngẫu nhiên trong vài mili giây đầu tiên.

Phần 4: Vòng lặp chính và xử lý nút nhấn

Loop:

```
JB flag_TUP, Do_Time
JNB P3.4, BtnC
JNB P3.5, BtnD
JNB P3.6, BtnE
SJMP Loop
```

Do_Time:

```
CLR flag_TUP
MOV A, MSEC
ADD A, #1
DA A
MOV MSEC, A
CJNE A, #00H, Time_Update_Dispatch
```

```
MOV A, SEC
ADD A, #1
DA A
MOV SEC, A
CJNE A, #00H, Time_Update_Dispatch
```

```
MOV SEC, #99H; Max hit
MOV MSEC, #99H
CLR flag_RUN
SETB flag_STP
```

Time_Update_Dispatch:

```
LCALL Update_Buffer
SJMP Loop
```

BtnC; +1s

```
JNB P3.4, $; Wait for release
MOV A, SEC
CJNE A, #99H, Add_1s
```

```

        SJMP Loop
Add_1s:
        ADD A, #1
        DA A
        MOV SEC, A
        LCALL Update_Buffer
        SJMP Loop

BtnD:: -1s
        JNB P3.5, $; Wait for release
        MOV A, SEC
        JZ Loop; Skip if 0
        ADD A, #99H; Add 99 is sub 1 in BCD
        DA A
        MOV SEC, A
        LCALL Update_Buffer
        SJMP Loop

BtnE:: Toggle speed
        JNB P3.6, $; Wait for release
        CPL flag_SPD
        MOV TREL, #2
        JNB flag_SPD, Loop
        MOV TREL, #1
        SJMP Loop

```

- **Vòng lặp Loop:** Liên tục kiểm tra theo thứ tự ưu tiên: flag_TUP từ ISR trước, sau đó mới đến ba nút nhấn. Nếu không có cò nào bật và không có nút nào được nhấn, CPU cứ lặp lại vòng kiểm tra liên tục mà không làm gì thêm, chờ đến khi có sự kiện xảy ra.
- **Đếm giờ (Do_Time):** Khi ISR set flag_TUP, main loop nhảy vào đây, xóa ngay flag_TUP để không xử lý lặp lại, sau đó cộng MSEC thêm 1. Cặp lệnh ADD A, #1 và DA A đảm bảo kết quả luôn là số BCD hợp lệ. Nếu MSEC tràn từ 99 về 00 thì SEC được cộng tiếp theo cùng cách. Nếu SEC cũng tràn, tức đồng hồ vừa vượt qua 99.99s, hệ thống lập tức khóa cứng giá trị ở 99.99, tắt flag_RUN để dừng đếm và bật flag_STP để LED báo hiệu sáng lên.
- **BtnC (+1s) / BtnD (-1s):** Vòng JNB pin, \$ giữ CPU chờ tại chỗ cho đến khi nút được nhả ra, đảm bảo mỗi lần nhấn chỉ xử lý đúng một lần. BtnC kiểm tra nếu SEC đã là 99 thì bỏ qua, không cho vượt giới hạn. BtnD kiểm tra nếu SEC đang là 00 thì bỏ qua,

không cho xuống âm. Sử dụng kỹ thuật cộng với phần bù 10 của BCD (thêm 99) kết hợp lệnh DA A để thực hiện phép trừ 1, đảm bảo tính tuần hoàn của bộ đếm.

- **BtnE (toggle 1x/2x):** Đảo flag_SPD để ghi nhớ trạng thái, sau đó luôn gán TREL = 2 trước. Nếu flag_SPD = 0 tức vừa tắt chế độ nhanh (2x) thì giữ nguyên TREL = 2 và thoát. Nếu flag_SPD = 1 tức vừa bật chế độ nhanh (2x) thì ghi đè TREL = 1.

Phần 5: Cập nhật buffer hiển thị:

Update_Buffer:

; Extract SEC tens

MOV A, SEC

SWAP A

ANL A, #0FH

MOV 40H, A

; Extract SEC units

MOV A, SEC

ANL A, #0FH

MOV 42H, A

; Extract MSEC tens

MOV A, MSEC

SWAP A

ANL A, #0FH

MOV 44H, A

; Extract MSEC units

MOV A, MSEC

ANL A, #0FH

MOV 46H, A

; Handle STOP LED on P1.7

JNB flag_STP, Clr_Stop_LED

RET; flag_STP=1: bit7=0 -> LED ON

Clr_Stop_LED:

ORL 40H, #80H; bit7=1 -> LED OFF

ORL 42H, #80H

ORL 44H, #80H

```
ORL 46H, #80H
```

```
RET
```

Tách từng chữ số BCD từ SEC và MSEC rồi ghi vào các ô chẵn của buffer. SWAP A và ANL A, #0FH lấy nibble cao (chữ số hàng chục), ANL A, #0FH lấy nibble thấp (chữ số hàng đơn vị). Bit 7 của mỗi ô dùng để điều khiển LED báo STOP trên P1.7: nếu flag_STP = 0 thì set bit 7 lên 1 (tắt LED), nếu flag_STP = 1 thì giữ bit 7 = 0 (bật LED).

Phần 6: Ngắt Timer 0 để quét LED và đếm giờ

T0_ISR:

```
PUSH ACC
```

```
PUSH PSW
```

```
CPL P2.5;
```

```
; Reload timer while running
```

```
MOV A, TL0
```

```
ADD A, #078H; Period LSB for 12MHz 5ms
```

```
MOV TL0, A
```

```
MOV A, TH0
```

```
ADDC A, #0ECH; Period MSB
```

```
MOV TH0, A
```

```
; Ghosting fix
```

```
ANL P2, #0E0H
```

```
MOV R0, SCAN_PTR
```

```
MOV P1, @R0; Output digit data
```

```
INC R0
```

```
MOV A, @R0; Load LED select mask
```

```
ORL P2, A; Enable current LED (1 instr vs 3)
```

```
INC R0
```

```
CJNE R0, #48H, T0_Ptr
```

```
MOV R0, #40H
```

T0_Ptr:

```
MOV SCAN_PTR, R0
```

```
JNB flag_RUN, T0_End
```

```
DJNZ TCNT, T0_End
```

```
MOV TCNT, TREL
```



```

SETB flag_TUP

T0_End:

    POP PSW
    POP ACC
    RETI

END

```

- **PUSH/POP ACC, PSW:** ISR dùng ADD/ADDC để reload timer, các lệnh này làm thay đổi thanh ghi ACC và cờ CY trong PSW. Nếu không lưu lại trước khi vào ISR, khi thoát ra main loop sẽ thấy ACC và CY đã bị ISR ghi đè. Lệnh DA A trong Do_Time đọc CY để hiệu chỉnh BCD, nếu CY sai thì kết quả đếm giờ sai. PUSH lưu giá trị gốc lên stack trước khi ISR làm việc, POP phục hồi lại trước khi RETI để main loop tiếp tục.
- **CPL P2.5:** Đảo chân P2.5 mỗi lần ngắt. Cứ 2 lần ngắt liên tiếp thì chân này hoàn thành một chu kỳ cao / thấp, tạo sóng vuông 10ms trên oscilloscope, tương đương chu kỳ ngắt Timer 0 là 5ms.
- **Reload timer:** Thay vì dừng timer, nạp lại rồi chạy tiếp, code cộng thẳng 0xEC78 vào TH0:TL0 trong khi timer đang chạy. Cách này tự động bù số chu kỳ đã trôi qua từ lúc ngắt xảy ra đến lúc code reload, nên chu kỳ 5ms được giữ chính xác.
- **Chống ghosting:** Trước khi chuyển sang digit mới, ANL P2, #0E0H tắt toàn bộ chân chọn LED về 0. Nếu không làm bước này, khoảnh khắc P1 đang chuyển dữ liệu sang digit mới trong khi LED cũ vẫn còn sáng sẽ khiến digit sai bị hiện ra trong tích tắc, mắt người nhìn thấy như LED bị nhòe hoặc hiện số lạ.
- **Quét LED:** Con trỏ R0 trở vào ô chẵn, đọc digit data ra P1, tăng lên 1 trở vào ô lẻ, đọc mask ORL vào P2 để bật đúng LED tương ứng, tăng thêm 1 sang cặp kế tiếp. Khi R0 vượt qua 47H thì wrap về 40H bắt đầu lại từ LED 1. Toàn bộ quá trình chỉ là tăng con trỏ, không cần biết đang ở LED nào vì cấu trúc RAM xen kẽ đã lo sẵn thứ tự.
- **Đếm giờ:** Phần này chỉ chạy khi flag_RUN = 1, tức đồng hồ đang chạy. Mỗi lần ngắt DJNZ TCNT trừ 1, khi TCNT về 0 mới nạp lại TREL vào TCNT và set flag_TUP để báo cho main loop biết đã đủ thời gian, cần tăng MSEC.

III. Exercise:

Compare the difference between edge interrupt and level interrupt?

Tiêu chí	Ngắt theo mức (Level Interrupt)	Ngắt theo cạnh (Edge Interrupt)
Bản chất kích hoạt	Vi điều khiển nhận diện ngắt khi tín hiệu ở một mức điện áp cụ thể.	Vi điều khiển nhận diện ngắt khi có sự thay đổi trạng thái điện áp.

Cấu hình	Bit ITx trong thanh ghi TCON = 0.	Bit ITx trong thanh ghi TCON = 1.
Đặc điểm đáp ứng	Ngắt sẽ được kích hoạt lại liên tục nếu tín hiệu vẫn giữ ở mức kích hoạt sau khi kết thúc trình phục vụ ngắt (lệnh RETI).	Ngắt chỉ được kích hoạt một lần duy nhất cho mỗi lần chuyển trạng thái (sườn lên hoặc sườn xuống). Muốn ngắt lại, tín hiệu phải chuyển trạng thái một lần nữa.
Ưu điểm	Đơn giản, phản hồi miễn là điều kiện còn tồn tại. Phù hợp khi thiết bị ngoại vi cần được phục vụ cho đến khi giải quyết xong vấn đề.	Tránh hiện tượng kích hoạt ngắt nhiều lần không mong muốn với cùng một sự kiện.
Nhược điểm	Nếu không xử lý phân cứng/phần mềm kịp thời để thay đổi mức tín hiệu trước khi kết thúc ngắt, sẽ bị kẹt trong vòng lặp ngắt liên tục (treo hệ thống).	Có thể bỏ lỡ tín hiệu ngắt nếu xung quá hẹp (xảy ra nhanh hơn chu kỳ lấy mẫu của vi điều khiển).
Ứng dụng tiêu biểu	Giao tiếp truyền dữ liệu liên tục như UART, I2C, SPI và các cảm biến duy trì trạng thái lâu như cảm biến mức nước.	Xử lý nút bấm, đếm xung và bắt các sự kiện chuyển đổi trạng thái xảy ra chớp nhoáng.

IV. References

- TS. Vũ Đức Lung, TS. Lê Quang Minh, ThS. Phan Đình Duy, *Giáo trình Vi điều khiển*, Trường Đại học Công nghệ Thông tin - ĐHQG TP.HCM, 2015
- Link video:
<https://drive.google.com/file/d/1H5FugA5p0kYzUEeznIpQfDrAinv6b3o8/view?usp=sharing>